

Computer Programming Summer Interest Group

Session 9 / Classes

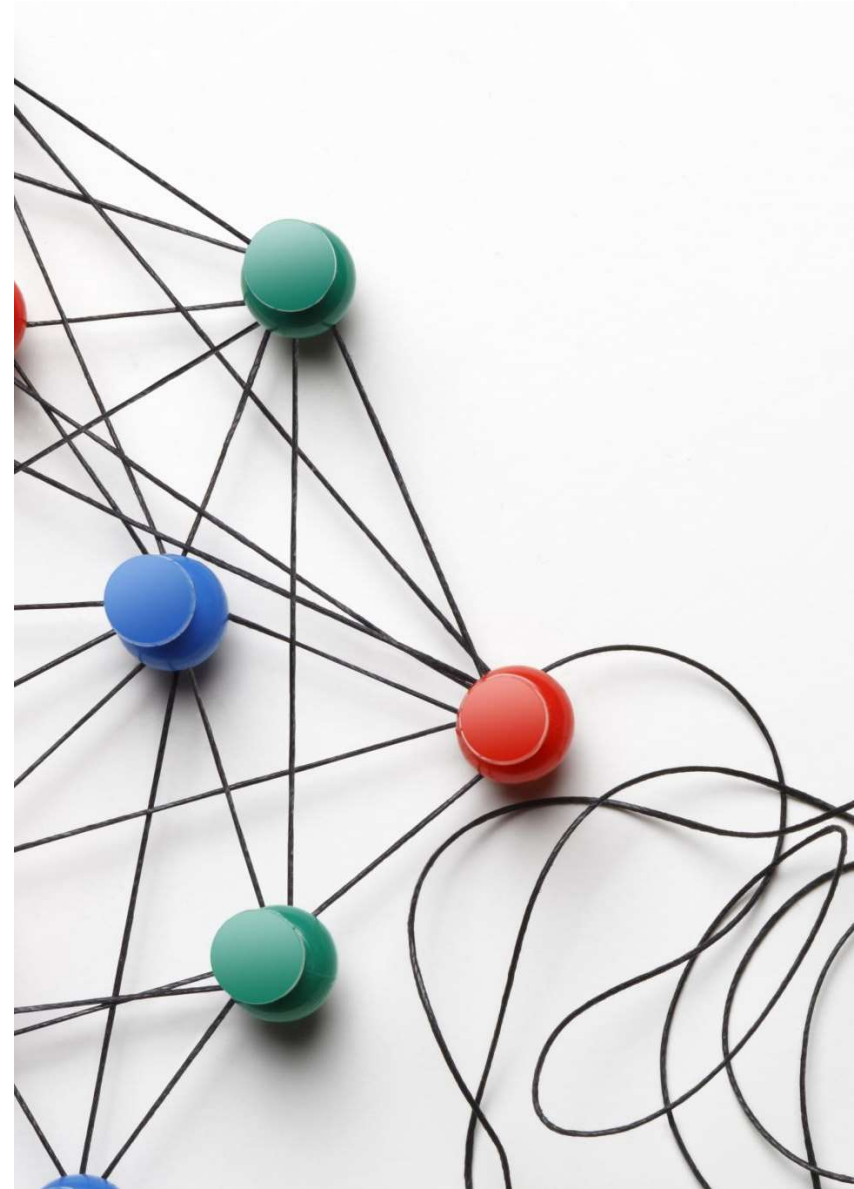
8/13/2025

1 Corinthians 12:15-20

Now if the foot should say, “Because I am not a hand, I do not belong to the body,” it would not for that reason stop being part of the body. And if the ear should say, “Because I am not an eye, I do not belong to the body,” it would not for that reason stop being part of the body. If the whole body were an eye, where would the sense of hearing be? If the whole body were an ear, where would the sense of smell be? But in fact God has placed the parts in the body, every one of them, just as he wanted them to be. If they were all one part, where would the body be? As it is, there are many parts, but one body.

Objectives

- By the end of this session, you will be able to:
 - Understand the concept of classes and objects in Python.
 - Create and use classes with attributes and methods.
 - Differentiate between instance variables and class variables.
 - Use constructors (`__init__`) to initialize objects.
 - Use “dunder” methods to tailor class behavior.
 - Write simple programs using classes.





What is a Class?



- One of the core components of object-oriented programming
- It binds data and functions intended to operate upon that data.
 - Attributes: The data elements of a class
 - Methods: The functions that operate on the class data
- A class definition just defines the attributes and methods of a class
- An object is an “instantiation” of the class
 - It’s an actual value associated with a variable that can be acted upon

Example of a Python Class Definition

- The keyword 'class' is used to designate the start of the class definition
 - This class has the name "Automobile"
- The class definition has two methods
 - `__init__` - Special method called an initializer or constructor
 - `update_mileage` - Method for updating automobile mileage
- The class definition has five attributes
 - `self.vin` - The vehicle identification number
 - `self.make` - The vehicle manufacturer
 - `self.model` - The vehicle model
 - `self.year` - The vehicle production year
 - `self.miles` - The vehicle mileage
- The initializer / constructor method is used to initialize an object which is an instantiation of the class
- All attributes and methods are externally accessible in Python
 - This is a difference from other object-oriented programming languages where attributes and methods can be restricted

```
class Automobile:
    def __init__(self,
                  vin: str,
                  make: str,
                  model: str,
                  year: int,
                  miles: int):
        self.vin = vin
        self.make = make
        self.model = model
        self.year = year
        self.miles = miles

    def update_mileage(self, miles: int):
        self.miles = miles
```

Class 'self' variable

- Note the use of the word 'self' throughout
- The first argument in a class's methods will reference the current object
 - By convention, this variable is 'self', though this is not required.
- Within a class this is used to designate the object related to the action being taken
- Other object-oriented programming languages (e.g. Java, C++) may use the word 'this' for this purpose.

```
class Automobile:
    def __init__(self,
                  vin: str,
                  make: str,
                  model: str,
                  year: int,
                  miles: int):
        self.vin = vin
        self.make = make
        self.model = model
        self.year = year
        self.miles = miles

    def update_mileage(self, miles: int):
        self.miles = miles
```

Object Instantiation

- To create a new instance, use the class name with the arguments of the `__init__` method.
 - This will execute the `__init__` method to initialize the resulting object
- Here we have two “instances” each representing different Automobile objects
 - Each has its own set of values for each object
- Other instances can be created in a similar manner.

```
camry = Automobile(  
    vin="4T1B11HK6JU635263",  
    make="Toyota",  
    model="Camry",  
    year=2018,  
    miles=125365)
```

```
thunderbird = Automobile(  
    vin="6Y81Z126801",  
    make="Ford",  
    model="Thunderbird",  
    year=1966,  
    miles=78345)
```


Accessing Attributes

- Use dot-notation to access attributes
 - Name of object and attribute separated by a period
- Attributes can be read or changed

```
print(f"{camry.miles=}")
```

```
camry.miles = camry.miles + 1285
```

```
print(f"{camry.miles=}")
```


Using class methods

- Use the dot-notation to access class methods
 - Name of object and name of method separated by a period
- Methods may:
 - Change the state of an object
 - Retrieve information about an object
 - Generally best to not mix purposes in a single method
- Methods should be well defined and focused on a single purpose
- Method names should be descriptive

```
print(f"{camry.miles=}")
```

```
camry.update_mileage(  
    miles=camry.miles + 1285)
```

```
print(f"{camry.miles=}")
```

“Dunder” Methods

- Methods with Double UNDERscores around the name
 - We’ve already seen `__init__`
- These have special purposes for Python classes implementing them
 - See handout for partial list
- `__call__` allows object to be called as a function
- `__add__` defines how to “add” two objects to get a third

```
class Particle:
    def __init__(self, pos: float, vel: float):
        self.pos = pos
        self.vel = vel

    def __call__(self, time: float = 0.0) -> float:
        return self.pos + time * self.vel

    def __add__(self, other: "Particle") -> "Particle":
        return Particle(
            (self.pos + other.pos) / 2.0,
            (self.vel + other.vel) / 2.0)
```

“Dunder” Methods Usage

- Use of `__init__` method
 - Referenced when creating objects ‘a’ and ‘b’ by calling `Particle(...)`
- Use of `__call__` method
 - Note references in the print statements where the `__call__` method is implicitly used
- Use of `__add__` method
 - Note “`c = a + b`” invokes use of the `__add__` method for the object “a”, where “b” is passes as the “other” argument

```
from particle import Particle

def main():
    a = Particle(pos=0.0, vel=10.0)
    b = Particle(pos=10.0, vel=20.0)
    c = a + b

    print(f"{a()} {b()} {c()}")
    print(f"{a(1.0)} {b(1.0)} {c(1.0)}")
    print(f"{a(-1.0)} {b(-1.0)} {c(-1.0)}")
    print(f"{a(100.0)} {b(100.0)} {c(100.0)}")

if __name__ == "__main__":
    main()
```

“Class” methods and attributes

- Refers to attributes or methods that are the same for all class objects
 - In other languages these are referred to as static methods or attributes
- Note definition of PI
 - Referenced as Circle.PI
- Note the “set_pi” method
 - It has a decorator above it “@staticmethod”
 - It does not have a “self” argument... it is for all Circle instances

```
import math

class Circle:
    PI = math.pi

    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return Circle.PI * self.radius**2

    def circumference(self) -> float:
        return 2 * Circle.PI * self.radius

    @staticmethod
    def set_pi(value: float):
        Circle.PI = value
```

“Class” methods and attributes usage

- Refers to attributes or methods that are the same for all class objects
 - In other languages these are referred to as static methods or attributes
- Note definition of PI
 - Referenced as Circle.PI
- Note the “set_pi” method
 - It has a decorator above it “@staticmethod”
 - It does not have a “self” argument... it is for all Circle instances

```
import circle

def main():
    c0 = circle.Circle(radius=1.0)
    c1 = circle.Circle(radius=2.0)

    print(f"{circle.Circle.PI=}")
    print(f"{c0.circumference()=} / {c0.area()=}")
    print(f"{c1.circumference()=} / {c1.area()=}")
    print()

    circle.Circle.set_pi(value=22 / 7)

    print(f"{circle.Circle.PI=}")
    print(f"{c0.circumference()=} / {c0.area()=}")
    print(f"{c1.circumference()=} / {c1.area()=}")
    print()

if __name__ == "__main__":
    main()
```



Activities

- Create a class for a 2-dimensional vector
 - The class should have two attributes
 - x as a float to serve as the X coordinate
 - y as a float to serve as the Y coordinate
 - Define the following methods
 - A dunder method to add two vectors and return their sum
 - A dunder method to subtract one vector from another
 - A standard method to return the length of the vector ($\sqrt{x^2 + y^2}$).
 - A standard method to return the direction of the vector ($\text{math.atan2}(y, x)$ – result is in radians)
- Write a main program showing usage of all of these methods.